

Penny: fundamentos del proyecto

Versión del documento: 1.0 · Fecha: 29 de septiembre de 2026

Este documento describe el problema que atiende Penny, propone la distribución de responsabilidades del equipo Scrum y delimita el alcance inicial. La asignación de personas es una **propuesta para el proyecto**; no afirma que esas personas hayan desempeñado ya esas funciones.

1. Antecedentes y problema de software del mundo real

Los gastos cotidianos suelen registrarse de forma dispersa o recordarse al final de la semana. Cuando falta un registro con importe, fecha y contexto, una persona puede conocer cuánto dinero le queda, pero no identificar fácilmente en qué categorías gastó, cuáles días concentraron sus compras ni cuánto sumaron los gastos pequeños.

Problema: una persona que quiere controlar sus gastos personales necesita registrar cada movimiento de manera rápida y consultar un resumen confiable de la semana sin sumar manualmente recibos, notas o movimientos separados. Para quien usa el celular como dispositivo principal, el registro también debe estar disponible sin depender de que una computadora o un servidor permanezcan encendidos.

Usuarios previstos: una persona que administra sus propios gastos, incluidos estudiantes y usuarios que desean entender sus hábitos semanales. El proyecto no presupone acceso a cuentas bancarias ni pretende sustituir asesoría financiera profesional.

Respuesta del sistema: Penny permite guardar concepto, importe en pesos mexicanos (MXN), fecha y una nota opcional; asignar una categoría automáticamente o elegirla; consultar una gráfica y una lista de gastos; y generar un reporte semanal de lunes a domingo con totales diarios y un PDF. La versión web conserva los datos en un archivo JSON local mediante Express. La aplicación Android los conserva en SQLite dentro del teléfono y funciona sin conexión. Los dos almacenamientos son independientes.

La descripción del problema es una **necesidad de diseño identificada para el proyecto**, no el resultado de una encuesta o investigación de campo documentada. Una validación posterior con usuarios podría medir tiempo de registro, frecuencia de uso y claridad de los reportes.

2. Roles y responsabilidades del equipo Scrum

En Scrum, las responsabilidades formales son **Product Owner**, **Scrum Master** y **Developers**. Las especialidades técnicas de la tabla distribuyen trabajo dentro del grupo de Developers; no crean roles formales adicionales de Scrum.

Integrante	Responsabilidad Scrum	Enfoque propuesto y decisiones a cargo
David Portillo	Product Owner	Ordenar el Product Backlog, definir el valor para el usuario, aclarar criterios de aceptación y decidir prioridades de alcance junto con el equipo.
Edwin Ramirez	Scrum Master	Facilitar la planificación, la revisión y la retrospectiva; ayudar a resolver impedimentos; cuidar la transparencia del avance y el uso del marco Scrum.
Josue Perez	Developer, interfaz web y experiencia de usuario	Implementar formularios, navegación, visualización de gastos, accesibilidad y adaptación a distintos tamaños de pantalla.
Christian Cisneros	Developer, API y reglas de negocio	Mantener rutas y validaciones de Express, modelo de gastos, categorías, persistencia web y cálculo del reporte semanal.
Abdel Gutierrez	Developer, Android y calidad	Integrar Capacitor y SQLite, preparar respaldos y APK, y coordinar comprobaciones funcionales en Android.

Responsabilidad compartida: todos los integrantes participan en la planificación del Sprint, estiman y ajustan el trabajo, revisan incrementos, mantienen la calidad y colaboran cuando una tarea cruza las especialidades propuestas. El Product Owner prioriza el producto; los Developers deciden cómo realizar el trabajo técnico. Esta distribución debe ser aceptada por el equipo al iniciar su trabajo.

3. Límites del alcance inicial y exclusiones

Alcance inicial del prototipo

La **línea base inicial** fue un prototipo web de uso personal y local. Incluía:

- 1 Registrar, editar y eliminar gastos con concepto, importe positivo, fecha, categoría y nota opcional.
- 2 Sugerir una categoría a partir del concepto y permitir la selección manual.
- 3 Conservar los registros entre sesiones en el archivo JSON del servidor local.
- 4 Mostrar los gastos de una semana seleccionada, con búsqueda en la lista, totales y gráfica proporcional por categoría.
- 5 Consultar un reporte de lunes a domingo, incluidos los días sin gastos, y descargarlo como PDF bajo demanda.
- 6 Validar los datos antes de guardarlos y mantener los cálculos monetarios en centavos enteros.

Límite operativo: una persona y una instancia local del servidor; el navegador necesita comunicarse con Express para usar la versión web. El alcance inicial no implicaba un servicio público alojado en Internet.

Exclusiones del alcance inicial

- Cuentas de usuario, autenticación, permisos y uso simultáneo por varias personas.
- Sincronización automática entre dispositivos, base de datos en la nube y funcionamiento web sin el servidor local.
- Conexión con bancos, tarjetas o sistemas de pago; importación automática de transacciones.
- Presupuestos, ingresos, deudas, predicciones o recomendaciones financieras.
- Conversión entre monedas; el prototipo contempla una sola moneda a la vez. La versión actual muestra MXN.
- Recordatorios, gastos recurrentes y generación automática del reporte dominical.
- Publicación en una tienda de aplicaciones y actualizaciones automáticas.

Evolución posterior del alcance

Después del prototipo inicial se añadió una aplicación Android instalable. Reutiliza la interfaz, pero procesa y guarda los gastos localmente en SQLite; incorpora importación y respaldo manual en JSON, y permite compartir el PDF. Esta ampliación **no convierte** la versión web y Android en un sistema sincronizado. La aplicación actual usa MXN para mostrar importes y mantiene sin conversión los valores numéricos registrados anteriormente.

Base de esta descripción

La delimitación se contrastó con `client/src/App.jsx`, `client/src/api.js`, `client/src/localExpenseStore.js`, `client/src/reportPdf.js`, `server/src/routes/expenseRoutes.js`, `server/src/models/Expense.js` y `server/src/storage/ExpenseStore.js`. Los roles son una propuesta organizativa, no información obtenida del repositorio.